Backend Testing & Logging

Session 2 after second freeze

Table of contents

Table of contents

01 What is Backend Testing

Backend Testing

- Backend testing is the process of **verifying the server-side components** of an application
- including the **application logic, APIs, server operations, and database**
- to ensure they function correctly, process data accurately, maintain data integrity, and handle requests reliably without errors.
- Its primary goal is to identify defects in the backend before they affect the application's functionality, performance, or users.

02 Types of Backend Testing

Types of Backend Testing

1. Manual Testing:

Manual testing is the process of verifying a software application by **executing test cases manually**, without using automation tools or scripts. A tester interacts with the application as a real user would, checking whether it behaves according to the requirements and reporting any defects found.

Types of Backend Testing

2. Automation Testing:

- is a software testing approach in which test cases are executed using tools and scripts instead of being performed manually.
- It helps teams test applications faster, improve accuracy, reduce repetitive effort, and support frequent releases in modern development environments.

types:

- *Executes test cases automatically using tools and scripts*
- *Used for repetitive, stable, and high-value test scenarios*
- *Commonly applied in regression, smoke, API, and CI/CD-based testing*

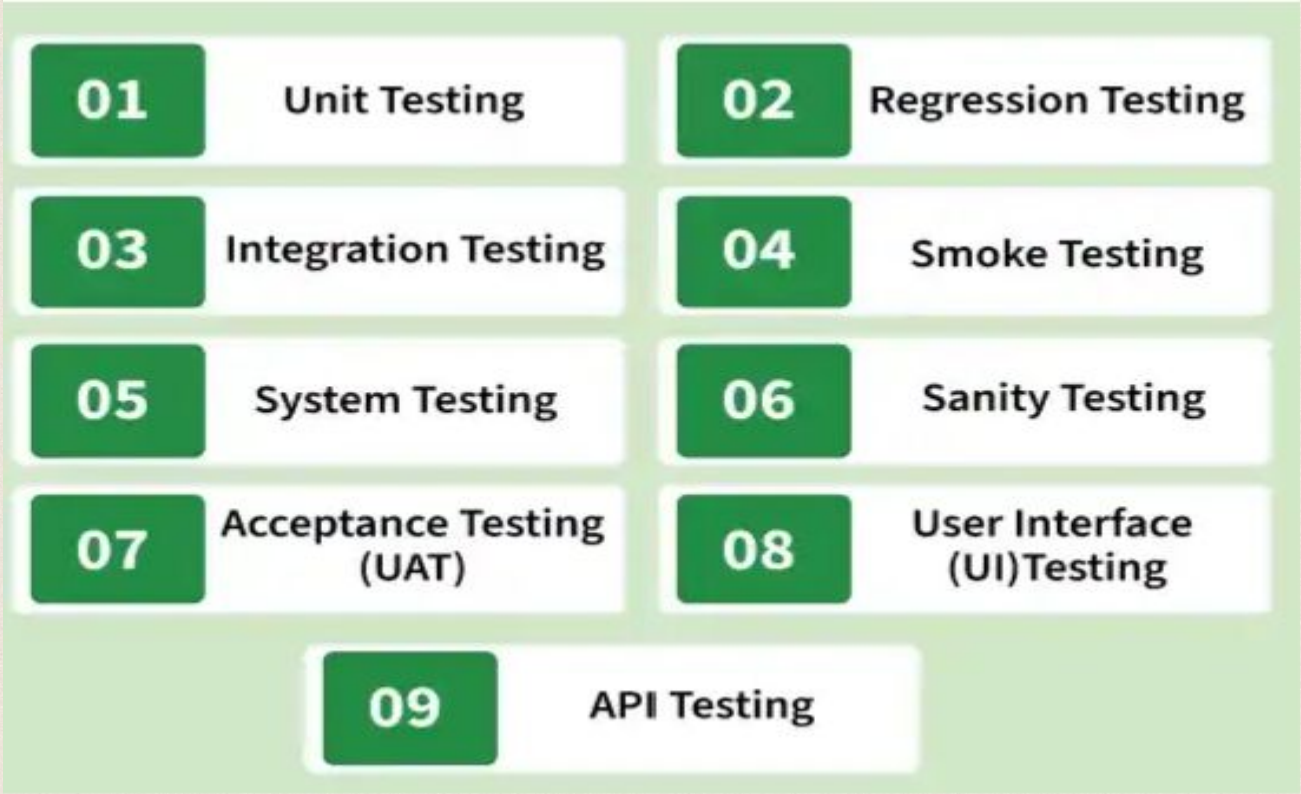
Types of Backend Testing

3. Functional Testing

- **Functional Testing** is a type of testing that verifies **what the software does**. It checks whether each feature behaves according to the specified functional requirements, regardless of how the code is implemented.

- In functional testing, the tester provides inputs, performs actions, and verifies that the outputs match the expected results. The internal code, algorithms, and implementation details are not the focus.

Types of Backend Testing



Types of Backend Testing

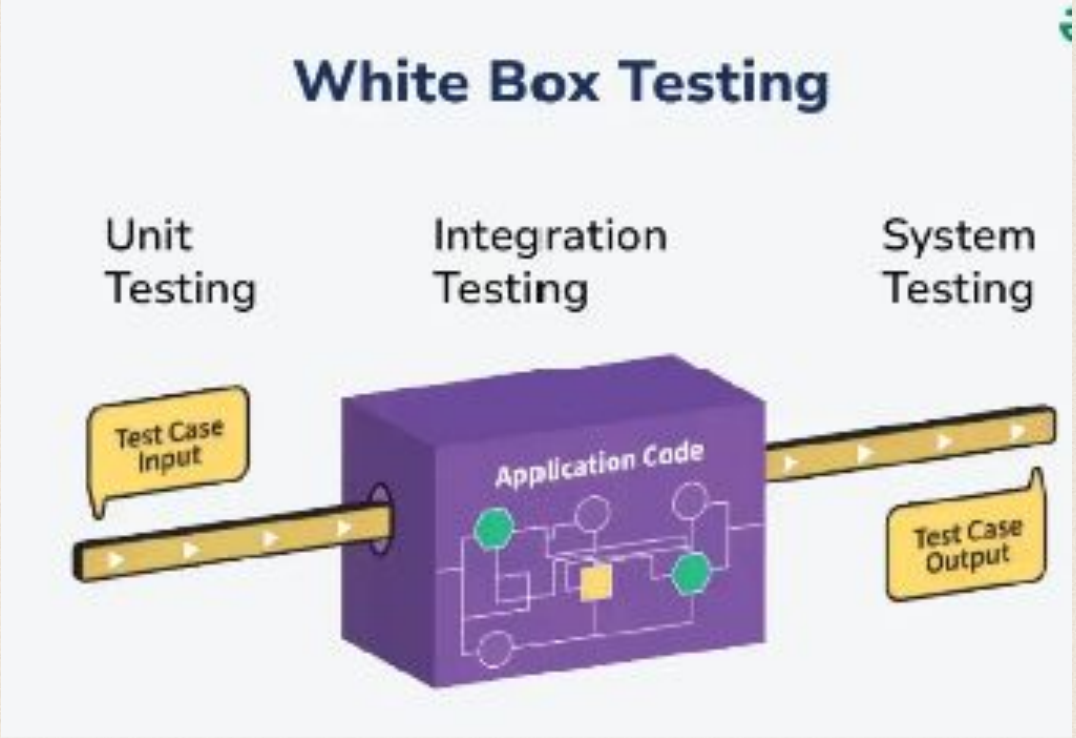
White Box Testing:

- is a software testing technique where the tester has **knowledge of the internal code and structure** of the application. It focuses on verifying the flow of logic, conditions, and code paths to ensure the program works correctly.
- test cases are derived from the **source code, control flow, branches, and implementation** details.

Examples:

- *A developer tests a login function by checking all code paths such as valid login, invalid password, and empty fields to ensure every condition works correctly.*

Types of Backend Testing



Types of Backend Testing

Black Box Testing:

- is a software testing technique where the **internal code or structure of the application is not known to the tester.** It focuses only on verifying the **functionality of the software** based on input and expected output. It ensures the system behaves as per user requirements.
- Test cases are derived from **requirements, specifications, and expected behavior.**

Examples:

A tester verifies a login page by entering username and password and checks whether the user is able to log in successfully. The internal code or logic is not considered while testing.

Types of Backend Testing

4. Non-Functional Testing

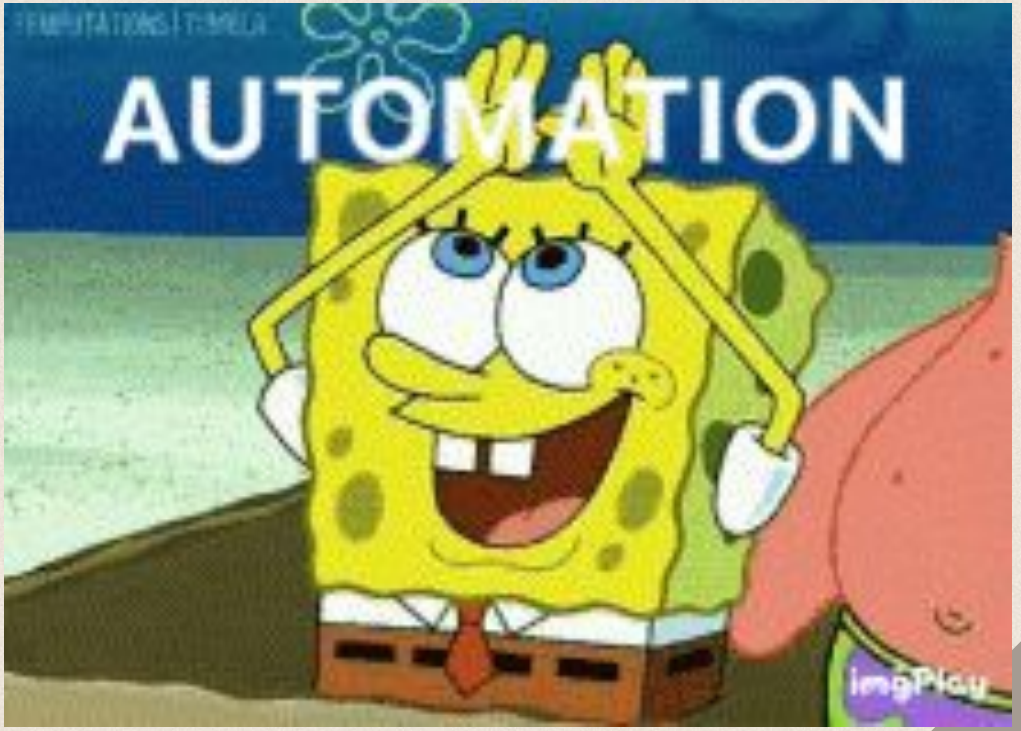
- is the process of performing load testing, stress testing, and checking minimum system requirements are required to meet the requirements.
- It will also detect risks, and errors and optimize the performance of the database.

Common types:

- ***Load Testing***: Measures performance under expected user load.
- ***Stress Testing***: Determines system behavior beyond normal capacity.

Types of Backend Testing

03 Automated VS Manual testing



Automated VS Manual testing

Manual	Automated
Needs a human	Automated
Takes time	Less time
Repeat for each new feature	Refactor with Confidence
Doesn't help with code	Write a better code

When to use both Automated & Manual testing

Automated

- Exploratory Testing
- Usability Testing
- Visual Validation
- Ad-hoc Testing

Manual

- Regression Testing
- Smoke Testing
- Data-Driven Testing
- Business-Critical Testing
- Deterministic Testing

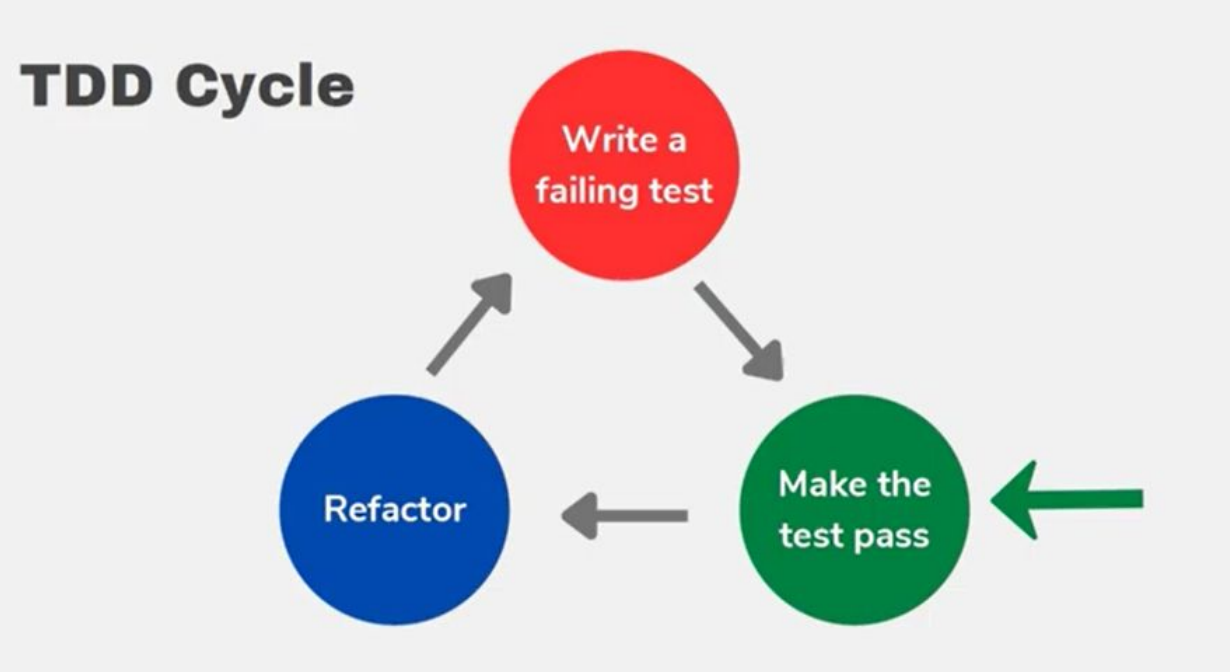
TDD & BDD

Test Driven Development (TDD):

- Test Driven Development (TDD) is a software development approach where automated **tests are created first** to define expected functionality before implementation begins.
- It ensures that every feature is developed only after defining its expected behavior through tests, which helps improve code quality and reduce bugs.

TDD & BDD

Test Driven Development (TDD):



TDD & BDD

Behavior Driven Development (BDD)

BDD is a software development approach that focuses on **defining the behavior of an application from the user's perspective**. It promotes collaboration between developers, testers, and business stakeholders to ensure that the final product meets expected requirements.

04 The Testing Pyramid

The Testing Pyramid–Key Layers of Testing

The **Testing Pyramid** is a software testing strategy that recommends how to distribute automated tests across different levels to achieve **fast feedback, high confidence, and lower maintenance cost**.

The idea is simple:

- **Write many fast, low-cost tests at the bottom.**
- **Write fewer complex, expensive tests at the top.**

The Testing Pyramid–Key Layers of Testing

1. Unit Testing:

test a **single functionality** and small units of code.
For unit testing, **isolation** is the key. Individual function is tested separately **without its external dependencies** .

Examples:

- **User authentication methods**: How does the login system respond to incorrect passwords or usernames? Does the system require the correct credentials?
- **Notification system**: Does the system provide accurate alerts when errors occur?

The Testing Pyramid–Key Layers of Testing

2. Integration Testing/Service testing:

Now that you've laid a strong foundation with unit testing, continue with the next phase of **integration (or service or component)** testing , which **checks the combination of the components**. They ensure that the modules, services, and systems will all work together seamlessly , with systems external dependencies.

Examples:

- **User Permissions:** Verify that when one user updates data, other authorized users see the updated information.
- **E-commerce Checkout:** Verify that login, cart, and payment services work together successfully.
- **Microservices Communication:** Verify that one service correctly communicates with other services (e.g., transaction → email).

The Testing Pyramid–Key Layers of Testing

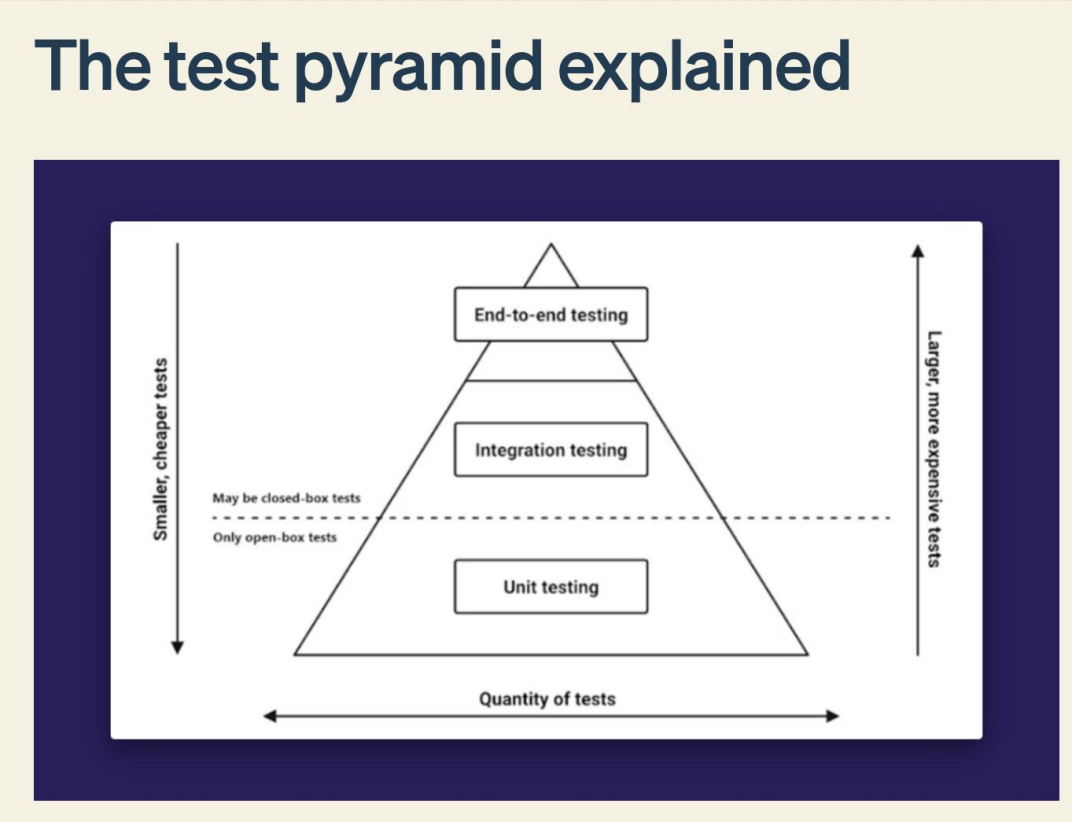
3. End-to-End (E2E) Testing User experience (UI) tests :

End-to-End (E2E) Testing validate the application **from the user's perspective.** These tests i**nteract with the fully deployed application** through its **user interface** to ensure that all integrated components (UI, backend, APIs, and database) work together correctly to deliver the expected user experience.

Examples:

- **Collaboration features:** Can team members work together on a project without running into bugs? Simulate different working situations.
- **Buying a product:** Simulate someone logging in, browsing, and purchasing a product.

The Testing Pyramid–Key Layers of Testing



05 Example of Testing Tools

Tools of testing :

Testing Frameworks

- **Jest** – Unit, integration, and API testing (most popular for Node.js).
- **Mocha** – Flexible JavaScript testing framework.
- **TestNG** – Java testing framework for unit and integration tests.

API Testing

- **Supertest** – Test Express/NestJS APIs.
- **Postman** – Test APIs manually or automate API collections.

CI/CD Automation

- **GitHub Actions** – Automatically runs your tests on every push or pull request.
- **Jenkins** – Automates building, testing, and deployment.

Tech Time !

06 What is Logging & Why?

What is Logging?

Logging :

- Logging is the process of automatically recording events that happen while your program is running.
- Think of it as the application writing down everything important it does.

Why is it called "Logging"?

The word comes from the English word log.

Originally, a logbook was literally a notebook used to record important events in chronological order.

So logging literally means:

"Keeping a log (a diary/journal) of everything important that happens."

What is Logging?

Imagine you're playing a game. Suddenly the game crashes.

- Without logs:
  - "It crashed."
- That's all you know.

- With logs:
  - Player joined
  - Loaded map
  - Connected to database
  - Spawned enemies
  - Saving progress...
  - ERROR: Database timeout
  - Game closed
- Now you know exactly where things went wrong.

What is Logging?

Logging is the process of recording important events that happen inside a running application so developers can monitor, debug, and understand what the system did.

What is Application Log?

- An application log is simply the collection of log messages generated by an application while it is running.

  - ***Logging*** → the action of writing records.
  - ***Log message*** → one individual record.
  - ***Application log*** → the complete file (or stream) containing all those records.

What is Application Log?

- Each line is a log message (log entry).
- All of these lines together form the application log.

```
2026-07-24 10:00:00 INFO  Server started
2026-07-24 10:00:05 INFO  Database connected
2026-07-24 10:01:10 INFO  User 152 logged in
2026-07-24 10:01:12 INFO  GET /profile completed (200)
2026-07-24 10:01:14 ERROR Database timeout
```

What is Application Log?

Why do we say application log?

- Because many things generate logs:
  - Operating System logs
  - Database logs (PostgreSQL, MongoDB)
  - Docker logs
  - Kubernetes logs
  - Application logs ← the logs generated by your code

Why is Logging important?

- Imagine it's 3:00 AM, Your backend serves 100,000 users.
- Suddenly someone calls you:
  - "The website isn't working."
- You weren't watching the server.
- The request happened hours ago.
- The user already closed the browser.
- You can't attach a debugger.
- You can't ask the server to replay the request.
- The only evidence left is the logs.
- That's why every production application logs events.

Why is Logging important?

- **Diagnostics / troubleshooting** — pinpoint exactly when and why something failed, especially in production where a debugger can't be attached.
- **Tracking & correlation** — follow a single request or user's journey across multiple services/components, and feed that trail into analytics or monitoring.
- **Security / Auditing** — a durable record of significant, security-relevant events; answers *"who did what, and when?"* — needed for compliance and for investigating incidents after the fact.
- **Performance monitoring** — logs of resource usage and response times help confirm a system is healthy, not just that it isn't crashing.

Why is Logging important?

- Think about CCTV cameras.
- They don't stop crimes.
- They record what happened so later you can investigate.
- Logs are the CCTV cameras of your backend.

07

What Every Program Should Log ?

What Every Program Should Log ?

Authentication & authorization — logins, logouts, failed login attempts, permission checks.
Changes — creation, updates, deletion of important data (who changed what).
Availability — application start-up, shutdown, restarts, health-check failures.
Resources — database connections opened/closed, connection pool exhaustion, external API call failures.
Threats / security — suspicious activity: repeated failed logins, malformed requests, rate-limit violations.

08 The Core Components of a Log Message

TMS - Notepad

File Edit Format View Help

```
INFO  - 2017-08-16T14:46:58 Searching for server cert in bindings for site "Default Web Site"
INFO  - 2017-08-16T14:46:58 Using server certificate thumbprint "A6528C9D0866F8405D451F876E124C9F91DE3DC3" - demomain.
INFO  - 2017-08-16T14:46:58 Registering Service Locators
INFO  - 2017-08-16T14:46:58 Database is configured
WARN  - 2017-08-16T14:47:02 No proxy server is specified
INFO  - 2017-08-16T14:47:02 Have 6 Console items
INFO  - 2017-08-16T14:47:02 Mapping console 62a7e338-e2cd-47ac-bde8-45b5edebd174 route "HelpDesk" from url "HelpDesk/{*queryva
INFO  - 2017-08-16T14:47:02 Mapping console f2e19194-9b58-40b9-b508-b9d9bdc461d4 static route "PrivilegeManager" from url "Pri
INFO  - 2017-08-16T14:47:02 Mapping console fc32a7ad-eb80-409c-81c6-587ba3ec8012 route "ResourceExplorer" from url "ResourceEx
INFO  - 2017-08-16T14:47:02 Mapping console fc32a7ad-eb80-409c-81c6-587ba3ec8012 route "ResourceExplorerAspx" from url "Resour
INFO  - 2017-08-16T14:47:02 Mapping console 352214ad-df09-4842-aa6e-bc47451b6c59 route "SecurityManager" from url "SecurityMan
INFO  - 2017-08-16T14:47:13 SignalR:Stream 0 : SQL notification change fired
INFO  - 2017-08-16T14:47:14 Platform Environment for Virtual App Default Web Site - /TMS (/TMS) Closing. Shutdown Reason Host
INFO  - 2017-08-16T14:47:14 SqlMessageBus got !immediate stop message, closing down SignalR processing.
INFO  - 2017-08-16T14:47:14 SignalR: SQL message bus disposing, disposing streams
WARN  - 2017-08-16T14:47:44 SqlMessageBus got immediate stop message.
INFO  - 2017-08-16T14:47:44 SignalR Stream 0 : SqlReceiver disposed
INFO  - 2017-08-16T14:53:18 Searching for server cert in bindings for site "Default Web Site"
INFO  - 2017-08-16T14:53:18 Using server certificate thumbprint "A6528C9D0866F8405D451F876E124C9F91DE3DC3" - demomain.
INFO  - 2017-08-16T14:53:18 Registering Service Locators
INFO  - 2017-08-16T14:53:18 Database is configured
INFO  - 2017-08-16T14:53:19 Have 6 Console items
INFO  - 2017-08-16T14:53:19 Mapping console 62a7e338-e2cd-47ac-bde8-45b5edebd174 route "HelpDesk" from url "HelpDesk/{*queryva
INFO  - 2017-08-16T14:53:19 Mapping console f2e19194-9b58-40b9-b508-b9d9bdc461d4 static route "PrivilegeManager" from url "Pri
INFO  - 2017-08-16T14:53:19 Mapping console fc32a7ad-eb80-409c-81c6-587ba3ec8012 route "ResourceExplorer" from url "ResourceEx
INFO  - 2017-08-16T14:53:19 Mapping console fc32a7ad-eb80-409c-81c6-587ba3ec8012 route "ResourceExplorerAspx" from url "Resour
INFO  - 2017-08-16T14:53:19 Mapping console 352214ad-df09-4842-aa6e-bc47451b6c59 route "SecurityManager" from url "SecurityMan
WARN  - 2017-08-16T14:53:20 No proxy server is specified
INFO  - 2017-08-16T14:54:29 SignalR:Stream 0 : SQL notification change fired
INFO  - 2017-08-16T14:55:40 AuditManager worker starting.
INFO  - 2017-08-16T14:55:44 SignalR:Stream 0 : SQL notification change fired
INFO  - 2017-08-16T14:56:55 SignalR:Stream 0 : SQL notification change fired
```

The Core Components of a Log Message

Context information — background detail that gives insight into the state of the app at the time of the message (e.g., request ID, user ID, module name) — an informative message that actually expresses what happened, not vague noise.

Timestamps — a specific piece of contextual information needed for tracking and correlating issues over time.

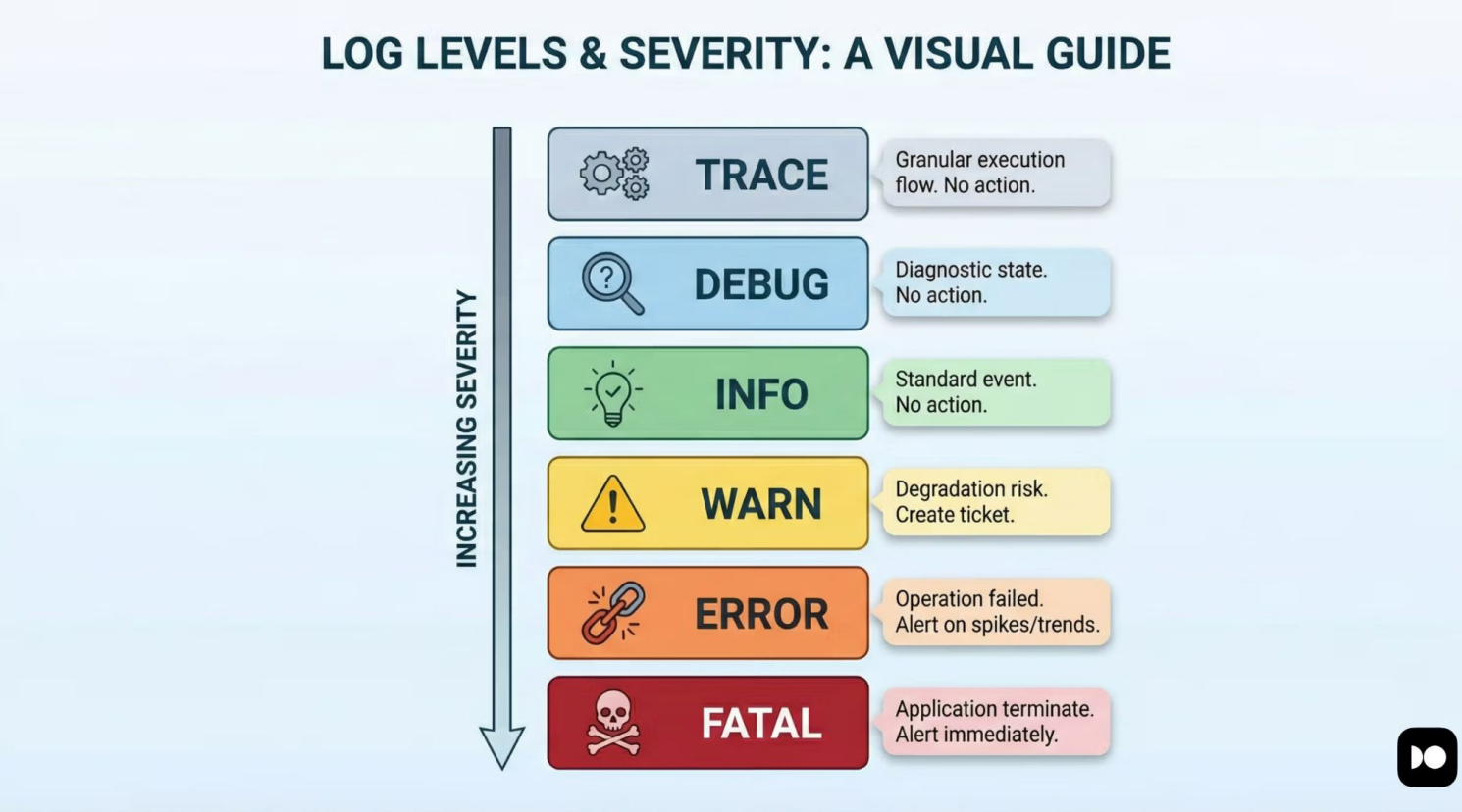
Log levels — labels that convey how important/severe an entry is, so entries can be filtered by seriousness.

09 Log Levels & Logging Ideals

Log Levels

- A **log level** or **log severity** is a piece of information telling how important a given log message is.
- It is a simple, yet very powerful way of distinguishing log events from each other.
- You can think of the log levels as a **way to filter** the critical information about your system state and the one that is purely informative.
- The log levels can help to reduce the **information noise and alert fatigue**.

Log Levels in Wiston



Log Levels in Wiston

Level	Meaning	When to Use	Example
TRACE	Extremely detailed execution flow.	Deep troubleshooting and tracing every step of execution. Rarely enabled in production.	Entering a function, every loop iteration, method calls.
DEBUG	Detailed information for developers.	Debugging issues during development. Usually disabled in production.	Database query executed, cache miss, variable values.
INFO	Normal application events.	Record expected application behavior.	Server started, user logged in, request completed.
WARN	Something unexpected happened, but the application can continue.	Situations that may require attention later.	Invalid input, failed login attempt, deprecated API used.
ERROR	An operation failed, but the application is still running.	Recoverable failures affecting a request or operation.	Database connection failed, payment API failed, file upload failed.
FATAL / CRITICAL	A severe error that prevents the application from continuing.	Unrecoverable failures that terminate the application.	Application crash, startup failure, out-of-memory error.

Backend Logging Ideals (Checklist)

1. Assign a Unique Request ID

or other non-sensitive identifiers, and redact sensitive data whenever possible.

2. Use Structured Logs

Store logs in a structured format (e.g., **JSON**) instead of plain text.

Structured logs are easier to search, filter, and analyze using logging tools.

3. Use Consistent Log Levels

Every log entry should include a severity level (e.g., **DEBUG, INFO, WARN, ERROR**).

Consistent levels allow developers to quickly filter logs and focus on the most important issues.

Backend Logging Ideals (Checklist)

4. Never Log Sensitive Information (PII)

Avoid logging personally identifiable information such as passwords, emails, phone numbers, or credit card details. Prefer logging user IDs or other non-sensitive identifiers, and redact sensitive data whenever possible.

5. Log with Context

Include useful context with every log, such as **Request ID, User ID, Route, IP Address, or Service Name.** Context makes logs much easier to understand and greatly simplifies debugging.

I already have console.log(). Why do I need logging?

Why Not Just use console.log?

`console.log` doesn't scale for a real backend:

- **No levels** — everything prints the same way; there's no way to filter "just show me errors" in production.
- **No structure** — output is plain text, hard to search, filter, or feed into a monitoring tool.
- **No destinations** — can't easily also write to a file, a database, or a remote log-collection service at the same time.
- **Performance under volume** — logging heavy traffic (1,000+ records) directly and synchronously can slow the whole application down. A proper logging library batches, buffers, and streams log writes in smaller, manageable chunks instead of blocking the main thread on every single write.
- **No safety net for errors** — logging a raw `Error` object with `console.log` often loses the stack trace and structured detail needed to actually debug it.

Tech Time !

Lets Kahoot !

Thank You !